Grupo 06
Análise do problema e divisão de tarefas

Inteligência Artificial I | Projecto prático 2026
Universidade Eduardo Mondlane - Engenharia Informática
Problema P6: “Engarrafamento na Baixa” (Rush Hour generalizado)

O que o grupo tem de construir

Um programa em Python que represente um parque de estacionamento, encontre uma sequência válida de deslocações para retirar o chapa X e compare estratégias de busca. Não basta resolver o tabuleiro: é necessário justificar a modelação, estudar heurísticas, medir o desempenho e explicar os resultados.

Membros no enunciado: Falaque, Dalton; Nhabinde, Alex; Nhabomba, Lino; Quive, Alexandre; Ricardo. O nome Ricardo aparece sem outros apelidos no documento e foi mantido assim.

Instância base: 6 × 6

```
A A . . O
P . . Q . O
P X X Q . O   -> saída
P . . Q . .
B . . . C C
B . R R R .
```

X é o veículo alvo, com 2 casas. A, B e C têm 2 casas; O, P, Q e R têm 3. A, X, C e R são horizontais; B, O, P e Q são verticais. Cada veículo conserva a sua orientação e não pode atravessar outros veículos ou obstáculos.

Como ler este documento

Os requisitos abaixo vêm das secções 2, 4.6 e 5 do enunciado. As representações, etapas e responsabilidades propostas são sugestões de organização. Este documento é um plano explicativo: não apresenta custos óptimos, contagens de estados ou resultados experimentais já calculados.

Fonte: Enunciado_Projecto_IA1_2026.pdf, sobretudo pp. 1-3, 7 e 11-12. Preparado em 9 de Novembro de 2026.

2 / Modelação e decisões essenciais

1. Estado e dados fixos

Uma representação compacta é uma tupla com a posição de referência de cada veículo, sempre na mesma ordem. Num veículo horizontal guarda-se a coluna da extremidade esquerda; num vertical, a linha da extremidade superior. Orientação, comprimento e linha/coluna fixa ficam na definição da instância, juntamente com dimensão, saída e obstáculos. Assim, a tupla é imutável e hashable e permite detectar estados repetidos sem copiar toda a grelha.

2. Acções e transições

Uma acção pode ser (veículo, sentido, k), com k casas de deslocação. Gerar todos os k legais, verificando todas as casas atravessadas. A transição altera apenas a posição desse veículo. Excluir k = 0. Verificar limites, colisões e obstáculos fixos; a excepção para X junto à saída depende da convenção de saída adoptada.

3. Dois objectivos de optimização

• **Menos movimentos:** cada deslocação de um veículo por k casas custa 1. BFS encontra o menor número de acções se cada acção for gerada correctamente.

• **Menor custo ponderado:** a mesma acção custa L × k. Por exemplo, um camião de comprimento 3 que anda 2 casas custa 6; um carro de comprimento 2 que anda 2 casas custa 4. UCS e A* devem usar estes custos.

Gerar apenas deslocações de uma casa faria a BFS minimizar casas percorridas, alterando o significado de “movimento”. Manter o mesmo conjunto de acções e mudar a função de custo permite uma comparação correcta entre os dois critérios.

4. Objectivo e convenção de saída

Definir se o objectivo é X atingir a borda direita ou sair completamente. Explicar se a deslocação final conta como acção e como é cobrada. O enunciado não detalha esta convenção; deve ficar documentada e, se necessário, ser confirmada com os docentes. A escolha afecta distância, custo óptimo e heurísticas.

5. O que se deve demonstrar

Generalizar para M × M até 8 × 8, incluindo obstáculos imóveis. Na base, verificar os óptimos dos dois critérios e explicar por que coincidem, como afirma o enunciado. Procurar outra instância em que os critérios escolham soluções diferentes. Para uma demonstração forte, mostrar que nenhum caminho de mínimo número de movimentos atinge o menor custo ponderado; soluções distintas por simples desempate não bastam.

3 / Implementação e heurísticas

Arquitectura obrigatória

Criar a classe abstracta Problema com estado_inicial(), accoes(s), resultado(s,a), custo(s,a,s2), e_objectivo(s) e heuristica(s,nome). Criar No com estado, pai, acção, g e profundidade, e caminho(no). As buscas recebem esta interface e não devem conhecer veículos ou grelhas.

Ficheiros mínimos: problema.py, dominio.py, buscas.py, heuristicas.py, gerador.py, experiencias.py, main.py, instancias/ com JSON, tests/ e README.md. Usar Python 3.10+ e implementar as buscas de raiz; não usar bibliotecas que já as resolvam.

Sete estratégias exigidas

• **BFS:** teste de objectivo na geração; explicar a perda de optimalidade no custo ponderado.

• **DFS:** controlo de ciclos no caminho e limite de profundidade configurável.

• **IDS:** profundidade progressiva e contagem de nós re-expandidos.

• **UCS:** heapq, melhor custo conhecido por estado e descarte/actualização de entradas obsoletas.

• **Gulosa:** executar com h1 e h2; prioridade determinada por h.

• **A* em grafo:** executar com h0 = 0, h1 e h2; prioridade g + h. Se uma heurística for inconsistente, tratar reaberturas para preservar as garantias aplicáveis.

• **IDA* com tabela de transposição:** busca em profundidade com limite em g + h; aumentar para o menor valor que excedeu o limite. Comparar com IDA* simples. Uma opção é limpar a tabela por iteração e guardar o menor g visto, podando reencontros sem melhoria. Justificar a poda e evitar reutilizar cegamente a tabela entre limites.

Heurísticas: estudar as dicas e ir além delas

O P6 sugere h1 = número de veículos distintos a bloquear X e h2 = h1 + bloqueadores dos bloqueadores. A regra geral exige pelo menos duas heurísticas não triviais diferentes das dicas ou que as melhorem. Por isso, analisar as duas sugestões e preparar versões melhoradas, deixando explícita a correspondência entre os nomes usados no código, nas experiências e no relatório.

h1 conta veículos, não casas ocupadas. Cada bloqueador directo tem de se mover, o que fornece um limite inferior quando toda a acção tem custo pelo menos 1. Já h2 precisa de uma definição rigorosa: evitar duplicações e assumir que todo bloqueador indirecto precisa de se mover pode estar errado se houver outra direcção para libertar o veículo. Não declarar admissibilidade sem prova.

Uma melhoria candidata é somar um limite inferior da deslocação de X ao custo mínimo de afastar cada bloqueador directo numa relaxação que ignora os restantes veículos. Deve tratar obstáculos, veículos que não podem sair da linha e a convenção de saída. É uma proposta a desenvolver e provar, não uma heurística já validada.

4 / Provas, gerador e experiências

O que verificar nas heurísticas

Admissibilidade: h(s) não pode exceder o custo óptimo restante h*(s). **Consistência:** em cada transição, h(s) ≤ c(s,a,s2) + h(s2), com valor zero nos objectivos. Fazer provas separadas para cada modelo de custo em que a heurística será usada. Se uma propriedade falhar, apresentar um estado/transição concreto e os respectivos valores.

Numa instância pequena, enumerar os estados e calcular h* por UCS a partir dos objectivos no grafo invertido ou por força bruta. Verificar automaticamente a admissibilidade em todos os estados e, como reforço, a consistência em todas as transições. Comparar dominância: h2 domina h1 apenas se nunca for menor no domínio relevante. Calcular também o factor de ramificação efectivo b* pela equação N + 1 = 1 + b* + ... + (b*)^d, indicando a convenção de N e d.

Gerador com dificuldade comprovada

A semente do Grupo 06 Laboral é **202606**. Gerar puzzles por busca para trás a partir de estados objectivo. A cadeia inversa garante uma solução, mas ter feito 15 deslocações aleatórias não prova que a solução mínima tem 15 movimentos. Certificar a distância óptima em número de acções com BFS ou outro método exacto justificado, e aceitar apenas candidatos com pelo menos 15 movimentos óptimos.

Se X for removido no estado final, é preciso definir a operação inversa de entrada de X. Também é possível trabalhar com uma configuração terminal em que X permanece representado, desde que essa convenção seja coerente com o teste de objectivo. Evitar duplicados e registar sementes, parâmetros, solução testemunha e dificuldade certificada.

Bateria experimental

• Pelo menos 10 instâncias por nível: pequena, média e grande, total mínimo de 30. O requisito de 15 movimentos do gerador pode tornar difícil obter casos “pequenos”; procurar casos pouco ramificados sem relaxar silenciosamente a exigência.

• Limite de 60 segundos e 2 GB por execução. Registar como falha o que exceder os limites, com o motivo; não omitir esses casos.

• Guardar custo da solução, comprimento do caminho, nós expandidos e gerados, máximo da fronteira, tempo com perf_counter e memória de pico com tracemalloc. Definir contagens equivalentes entre algoritmos; documentar como a memória do processo será limitada, pois tracemalloc não cobre toda a memória.

• Exportar CSV e pelo menos três gráficos: nós expandidos vs. tamanho em escala logarítmica; tempo vs. tamanho; custo obtido vs. óptimo. Não apresentar como óptimo um custo sem certificação.

Comparações próprias do P6: número de movimentos vs. custo ponderado; IDA* simples vs. IDA* com tabela; total de estados alcançáveis na base vs. fracção expandida por A* com h2. Para esta fracção, usar estados únicos expandidos / estados alcançáveis e reportar expansões totais à parte se houver reaberturas.

5 / Divisão aleatória das responsabilidades

Os nomes foram embaralhados aleatoriamente e associados aos cinco blocos abaixo. O sorteio define responsáveis principais; todos devem rever outras partes e saber explicar o projecto completo. A divisão é uma proposta de trabalho, não um registo de contribuições já feitas.

Nhabomba, Lino | Modelação e domínio

Definir estado, acções, custos e saída; implementar problema.py e dominio.py; carregar a base e JSON; validar colisões, obstáculos e dimensões até 8 × 8.

Entrega verificável: Interface e domínio prontos; diagrama da representação; testes de sucessores, imutabilidade e regras.

Falaque, Dalton | Buscas não informadas

Implementar BFS, DFS, IDS e UCS em buscas.py; reconstruir caminhos, tratar repetidos e limites; contar re-expansões do IDS e entradas obsoletas do UCS.

Entrega verificável: Quatro buscas genéricas; testes de optimalidade em casos pequenos, ciclos, limites e ausência de solução.

Nhabinde, Alex | Heurísticas e buscas informadas

Estudar e melhorar h1/h2; escrever provas ou contra-exemplos; implementar Gulosa e A*; validar admissibilidade e consistência numa instância pequena.

Entrega verificável: heuristicas.py; duas buscas; provas no relatório; comparação A* h0 com UCS e A* h1/h2.

Quive, Alexandre | IDA* e gerador

Implementar IDA* simples e com tabela de transposição; justificar podas; criar gerador.py com semente 202606 e certificação de 15+ movimentos; procurar caso em que os critérios divergem.

Entrega verificável: Duas versões comparáveis do IDA*; instâncias reproduzíveis; evidência da dificuldade e redução de nós.

Ricardo | Experiências, interface e integração

Implementar experiencias.py e main.py; centralizar métricas e limites; gerar CSV/gráficos e visualização; enumerar estados da base com apoio dos restantes; coordenar README e montagem do relatório.

Entrega verificável: Menu funcional; bateria de 30+ instâncias; três gráficos; contagem de estados; pacote final verificado.

Equilíbrio de trabalho: o responsável pela modelação apoia o gerador e a visualização depois de estabilizar o domínio; o responsável pelas buscas não informadas apoia a instrumentação. Cada membro escreve a sua parte do relatório e os testes da sua implementação. A integração e a revisão final são colectivas.

6 / Sequência de trabalho e lista de entrega

Ordem recomendada, sem inventar prazos

• **Etapa 1:** fechar a convenção de saída e movimento, definir a interface comum e carregar o tabuleiro base. Todos aprovam estes contratos antes de trabalhar em paralelo.

• **Etapa 2:** integrar domínio e UCS/BFS; usar problemas pequenos para conferir validade e custos. Implementar as restantes buscas e heurísticas sobre a mesma interface.

• **Etapa 3:** validar provas e testes; integrar IDA*, tabela e gerador. Só comparar desempenho depois de confirmar que todos resolvem exactamente o mesmo problema.

• **Etapa 4:** executar a bateria, analisar falhas, produzir gráficos e escrever a discussão. Guardar resultados reais, comandos e sementes para reprodução.

• **Etapa 5:** rever o pacote e ensaiar a defesa: cada pessoa explica e altera código que outra escreveu. Todos fazem commits identificáveis ao longo do trabalho.

Testes e interface

São exigidos pelo menos **15 testes pytest** para este grupo Laboral, incluindo sucessores conhecidos, instância sem solução, igualdade do custo de A* e UCS e admissibilidade. Acrescentar testes de deslocação por várias casas, colisão intermédia, obstáculos, saída, custos L × k, caminhos reconstruídos e equivalência de IDA* com/sem tabela em casos pequenos.

O menu deve permitir escolher instância de ficheiro ou gerada, algoritmo, heurística e limites, mostrando solução passo a passo e métricas. Usar ASCII ou matplotlib para visualizar o tabuleiro e o percurso.

Pacote final

• **IA_PROJECTO_GRUPO_06_LAB.zip**: relatório em PDF no template oficial; repositório completo com .git; CSV e gráficos; README com instruções de execução e reprodução.

• Relatório: introdução, revisão teórica, formulação e tamanho do espaço, heurísticas e provas, implementação, experiências e discussão, conclusões, referências, contribuições e declaração de uso de IA.

• Entrega indicada no enunciado: feuem.ia@gmail.com; assunto “IA: PROJECTO_GRUPO_06 (LAB)”. A data de entrega e as datas de defesa ainda estão por definir no documento. Confirmar com os docentes. Levar relatório impresso à defesa.

Preparação individual para a defesa

A defesa dura 20-25 minutos: demonstração, alteração ao vivo sem IA nem Internet e perguntas individuais. Todos devem explicar o custo óptimo da base, o número de estados alcançáveis, a fracção expandida por A*, a diferença entre os dois critérios e a utilidade da tabela de transposição. A nota final individual é a média de Trabalho e Defesa; defesa inferior a 8 limita a nota final a 9.

Uso desta análise: o enunciado permite apoio de IA com declaração. Registar que esta ferramenta ajudou a interpretar requisitos e a organizar tarefas; verificar as decisões e conclusões antes de as incorporar no trabalho final.